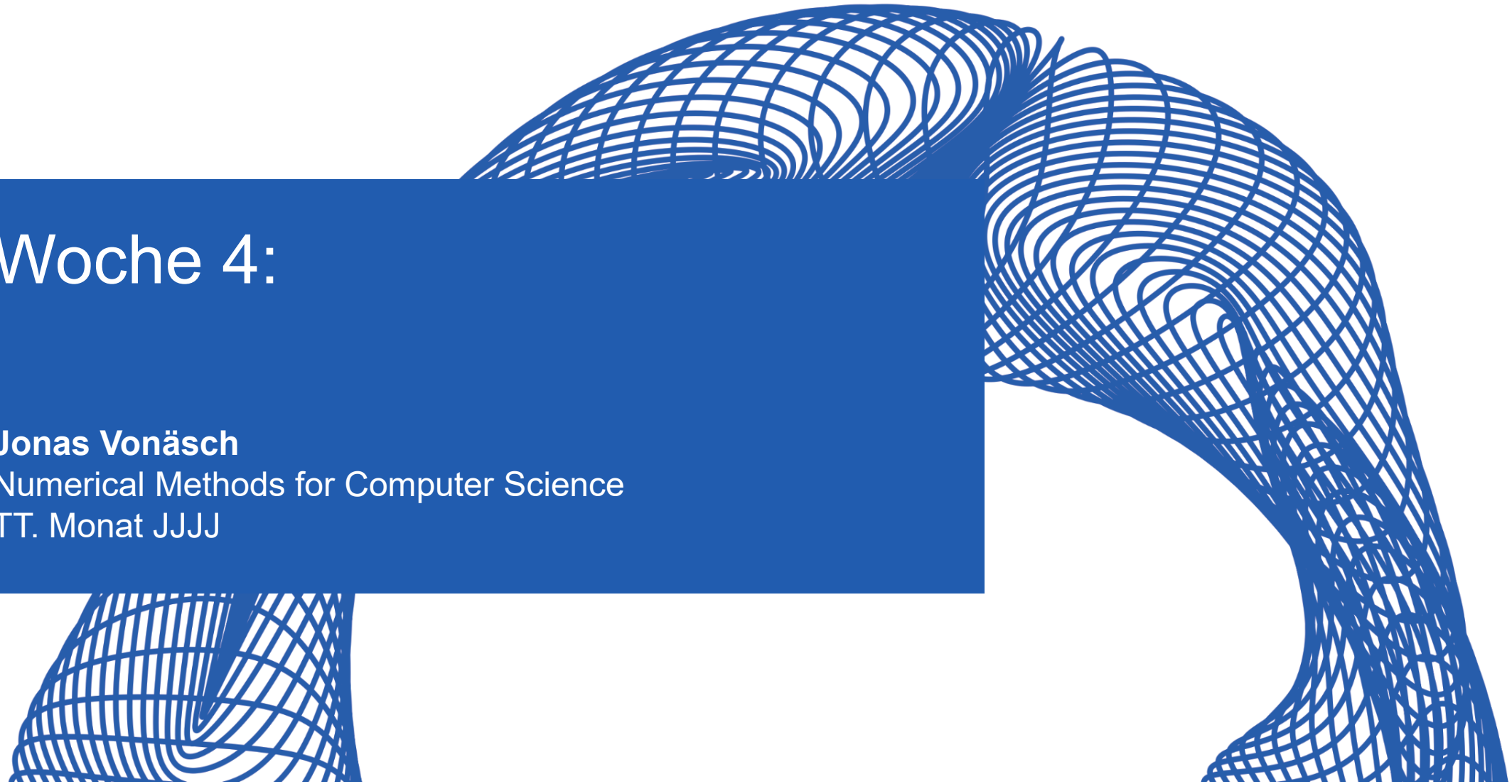
ETH zürich

Department Informatik



Woche 4:

Jonas Vonäsch

Numerical Methods for Computer Science

TT. Monat JJJJ

Agenda

1. Chebyshev-Interpolation & DCT
2. Stückweise lineare Interpolation
3. Kubisch Hermite Interpolation
4. Kubische Splines
5. Tafelbeispiel Quadratische Splines

Cosinus-Transformation

Cosinus-Transformation: Idee

Die Chebyshev-Polynome erinnern an Trig. Polynome. Chebyshev-Interpolation mit FFT lösen?

Erinnern wir uns an $T_n(x)$:

$$T_n(x) = \cos(n \arccos(x))$$

Und die Chebyshev-Knoten:

$$t_k = \cos\left(\frac{2k+1}{2(n+1)}\pi\right), k = 0, \ldots, n$$

Einsetzen liefert:

$$T_n(t_k) = \cos\left(n\pi\frac{2k+1}{2n+2}\right)$$

Das sieht nacch einem trig. Polynom aus! FFT möglich?

Cosinus-Transformation Fast-Forward

Wir schreiben die Chebyshev-Knoten um als:

$$t_k = \cos(2\pi\xi_k)\,, \qquad \xi_k = \frac{2k+1}{4(n+1)}$$

Die ξ_k stellen jetzt einen Teil unserer Stützstellen für die FFT dar.

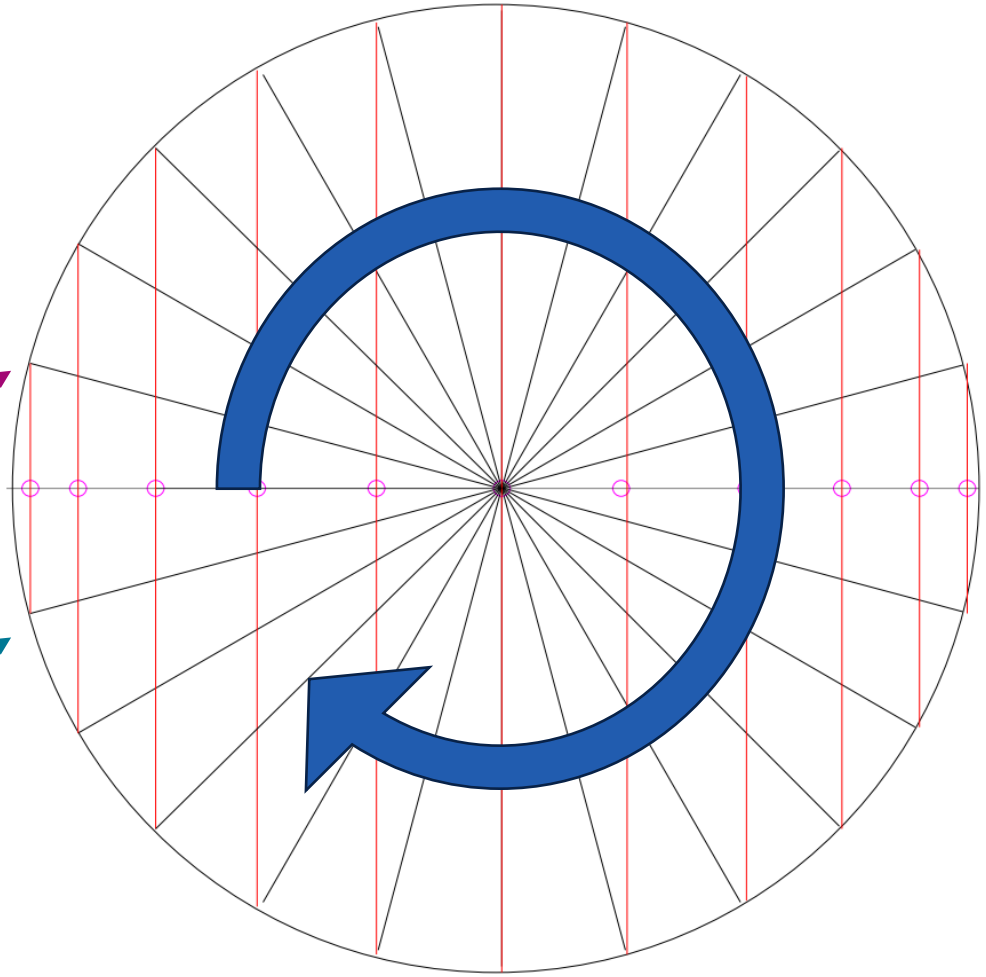
Allerdings sind sie äquidistant in $\left[0, \frac{1}{2}\right]$!

$$z = (y_0, \ldots, y_n,\ y_n, \ldots, y_0)$$

Wieso?

Intuition zur Spiegelung

$$z = (y_0, \ldots, y_n,\ y_n, \ldots, y_0)$$



$$x_0, \ldots, x_n, x_{n+1}, \ldots, x_{2n+2} = x_0, \ldots, x_n, x_n, \ldots, x_0$$

Zweite Hälfte der Knoten ist Weiterführung der ersten (selbe Messstellen, aber verkehrt)

Cosinus-Transformation: Vorgehen

- Wir messen f an den Stellen t_k und bekommen $y \to \mathrm{z}$
- Wir tun so, als hätten wir an ξ_k die Werte z gemessen
- Wir berechnen: $[\zeta_{k-n}] = F_{2n+2}^{-1}\left[\exp\left(\frac{\pi i n \ell}{n+1}\right) z_\ell\right]$
- Anhand von: $\zeta_j = c_j \exp\left(\frac{2\pi i j}{4(n+1)}\right)$ extrahieren wir c_j
- Wir extrahieren a_j aus c_j mit:

$$c_j = \begin{cases} \frac{1}{2} a_j & \text{für: } j = 1, \ldots, n \\ a_0 & \text{für: } j = 0 \\ \frac{1}{2} a_{-j} & \text{für: } j = -n, \ldots, -1 \end{cases}$$

- $\tilde{f}(t) = \sum_{j=0}^{n} a_j T_j(t)$

Chebyshev mit FFT: DCT

snippet.py

```python
def chebexp(y):
    # WICHTIG: gibt Koeffizienten für [-1, 1]
    n = y.shape[0] - 1

    t = np.arange(0, 2*n+2)
    z = np.exp(-np.pi * 1j * n/(n+1.0) * t) * np.hstack([y,y[::-1]])

    c = ifft(z)

    t = np.arange(-n, n+2)
    b = (np.exp(0.5j * np.pi/(n+1.0) * t) * c).real

    a = np.hstack([ b[n], 2 * b[n+1:2*n+1] ])
    return a
```

Stückweise Polynomiale Interpolation

Stückweise polynomielle Interpolation: Idee

Bei polynomieller Interpolation mit n Stützstellen, ergibt sich ein Polynom n-ten Grades!

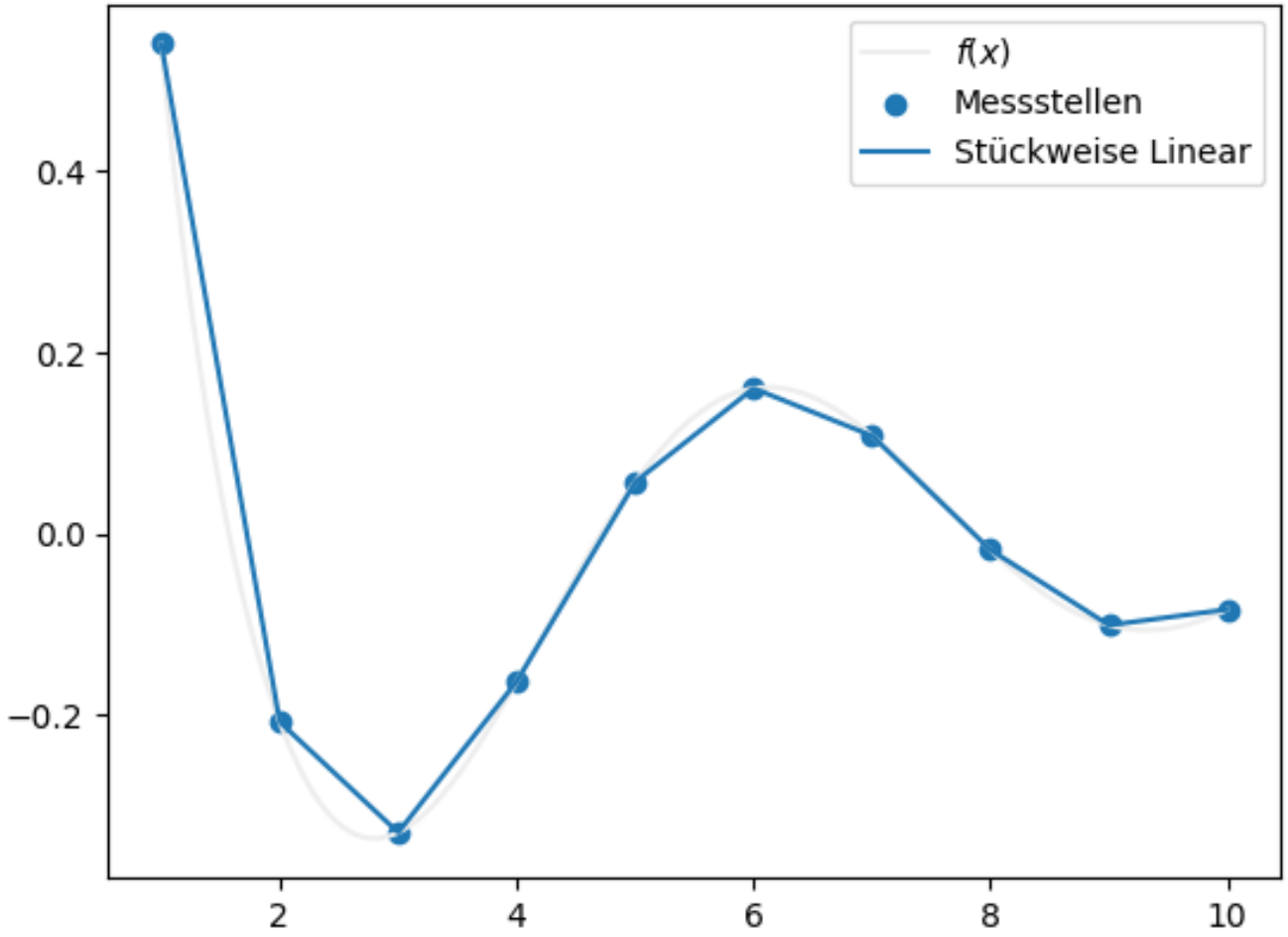
Runge-Phänomen, starke Schwingung!

Neuer Ansatz:

Wähle **breakpoints** $a = x_0 < x_1 < \ldots < x_N = b$

Interpoliere jeweils local mit einer Funktion f_j im Abschnitt $[x_j, x_j + 1]$

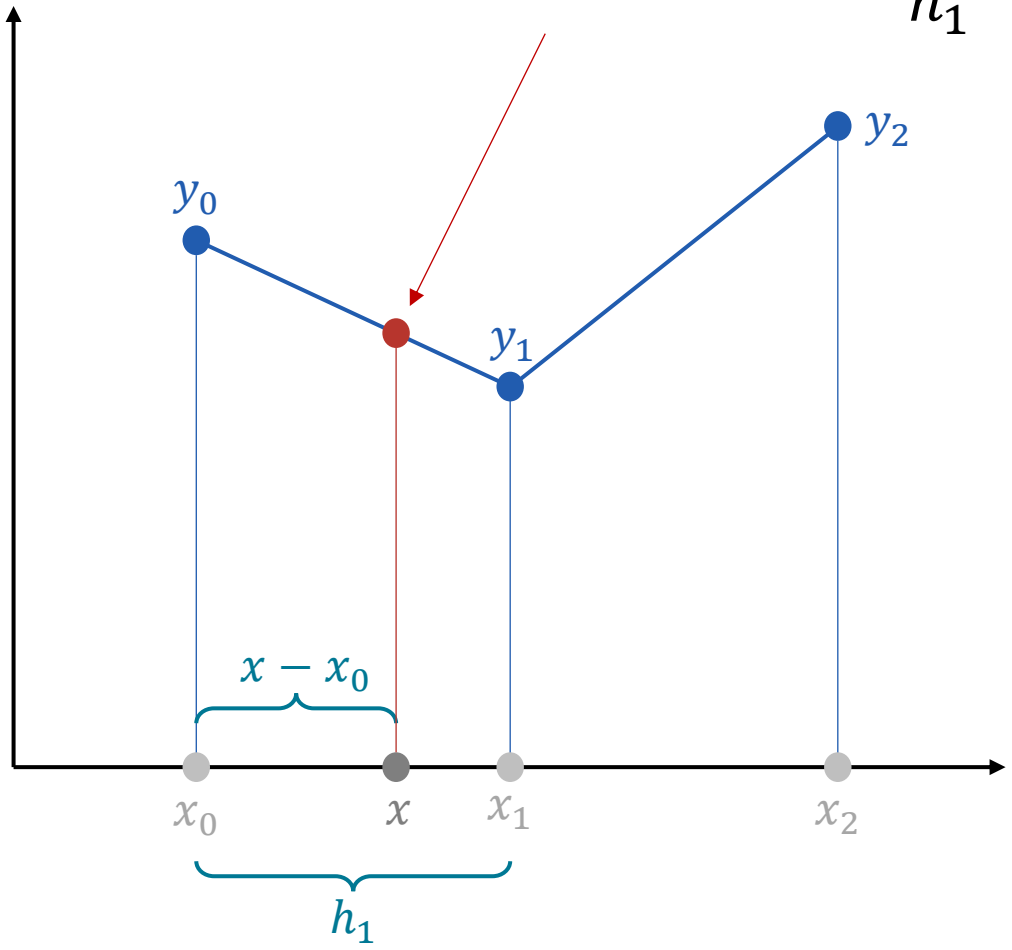
Stückweise polynomielle Interpolation: Illustration



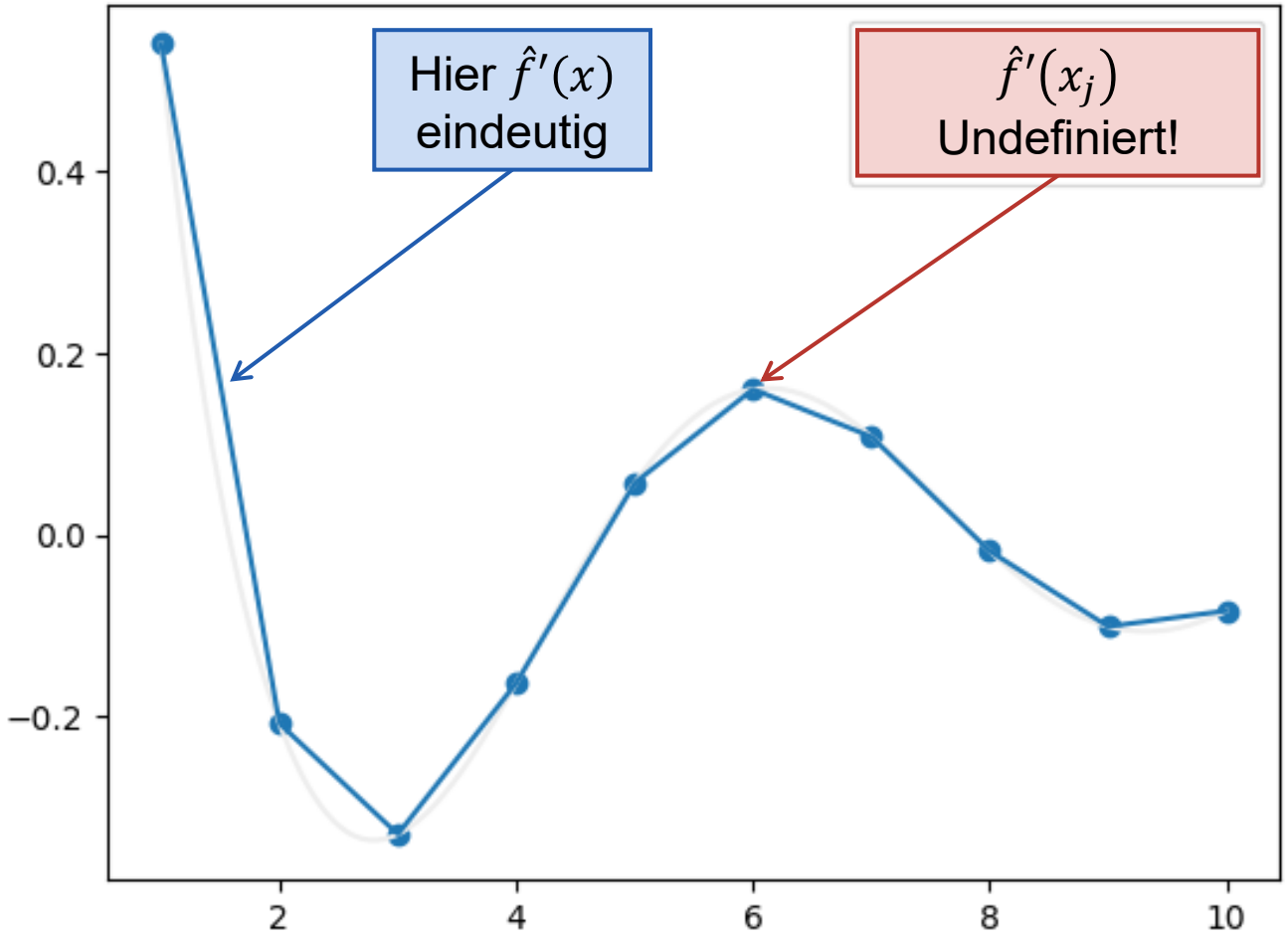
Stückweise Lineare Interpolation

Stückweise Lineare Interpolation

$$s_1(x) = y_0 \frac{x_1 - x}{h_1} + y_1 \frac{x - x_0}{h_1}$$



Stückweise Lineare Interpolation: Problem



Kubisch Hermite Interpolation

Kubisch-Hermite Idee

Wir wollen stetig differenzierbares $\hat{f}(x)$!

Betrachte Funktionen:

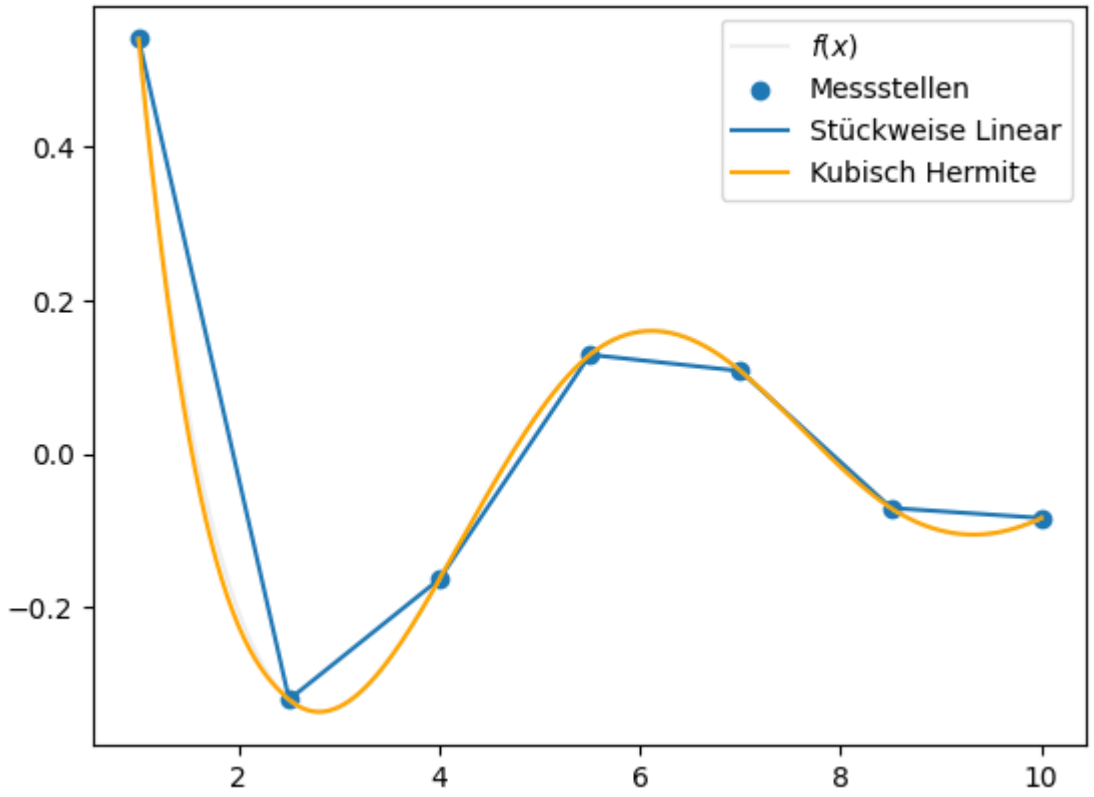
$$\phi(t) = 3t^2 - 2t^3 \qquad \psi(t) = t^3 - t^2$$

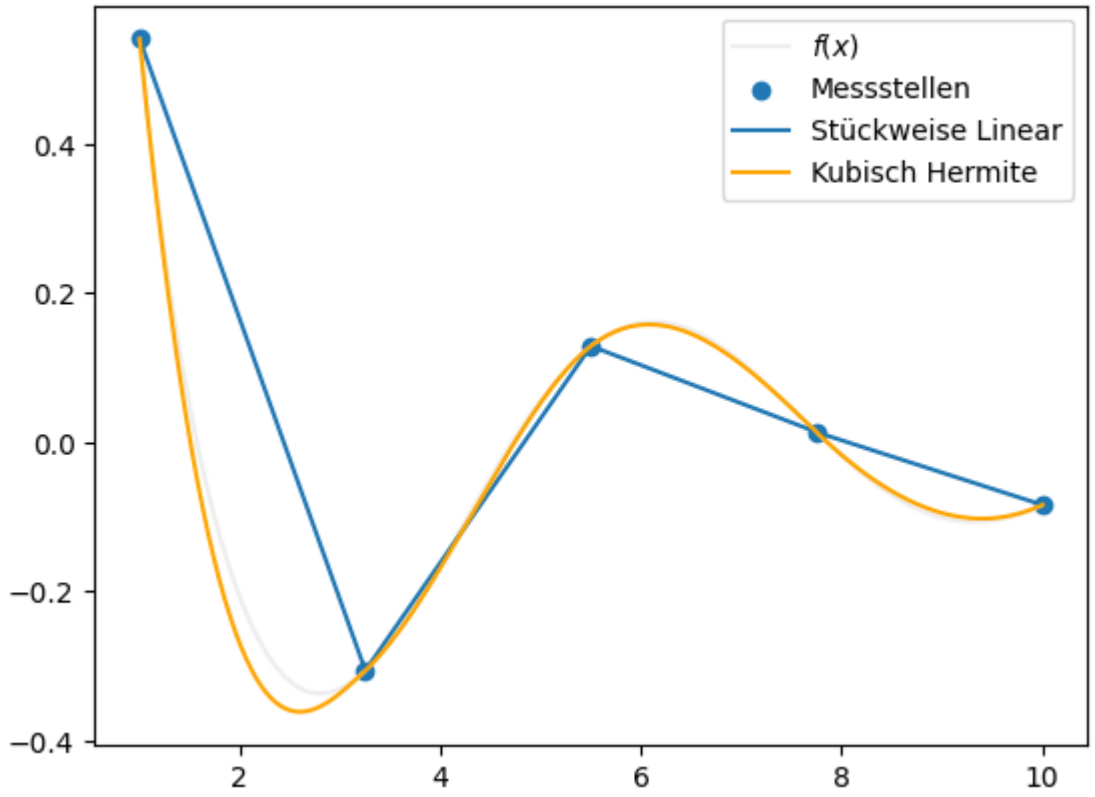
Wir beobachten:

$$\begin{aligned} \phi(0) &= 0 \\ \phi(1) &= 1 \\ \phi'(0) &= 0 \\ \phi'(1) &= 0 \end{aligned} \qquad \begin{aligned} \psi'(0) &= 0 \\ \psi'(1) &= 1 \\ \psi(0) &= 0 \\ \psi(1) &= 0 \end{aligned}$$

$\phi(t) \quad \phi(1-t) \quad \psi(t) \quad \psi(1-t)$ sind auf $t \in \{0,1\}$ orthonormal!

Kubisch Hermite: Plots





Kubisch Hermite Konstruktion

Angenommen $f(x_0), \ldots, f(x_N)$ und $f'(x_0), \ldots, f'(x_N)$ seien bekannt!

Im interval $[x_{j-1}, x_j]$ konstruieren wir

- $\hat{f}_j(x_{j-1})$: $\quad \phi\left(\frac{x_j - x}{h_j}\right) \quad = \phi(1-t)$
- $\hat{f}_j(x_j)$: $\quad \phi\left(\frac{x - x_{j-1}}{h_j}\right) \quad = \phi(t)$
- $\hat{f'}_j(x_{j-1})$: $\quad \psi\left(\frac{x_j - x}{h_j}\right) \quad = -h_j\psi(1-t)$
- $\hat{f'}_j(x_j)$: $\quad \psi\left(\frac{x - x_{j-1}}{h_j}\right) \quad = h_j\psi(t)$

$$t = \frac{x - x_{j-1}}{h_j}$$

$$\phi(0) = 0$$
$$\phi(1) = 1$$
$$\phi'(0) = 0$$
$$\phi'(1) = 0$$

$$\psi(0) = 0$$
$$\psi(1) = 0$$
$$\psi'(0) = 0$$
$$\psi'(1) = 1$$

Damit stimmen sowohl die Funktionswerte als auch Ableitungen jeweils in allen Stützstellen mit f überein.

Kubisch Hermite Formel

$$p_j(x) = y_{j-1}\phi(1-t) + y_j\phi(t)$$

Passt genau auf die Funktionswerte y_{j-1}, y_j, die Ableitungen sind hier 0.

$$q_j(x) = -y'_{j-1}h_j\psi(1-t) + y'_j h_j\psi(t)$$

Passt genau auf die Ableitungen y'_{j-1}, y'_j, die Funktionswerte sind hier 0.

$$f_j(x) = p_j(x) + q_j(x)$$

Kubisch Hermite Formel

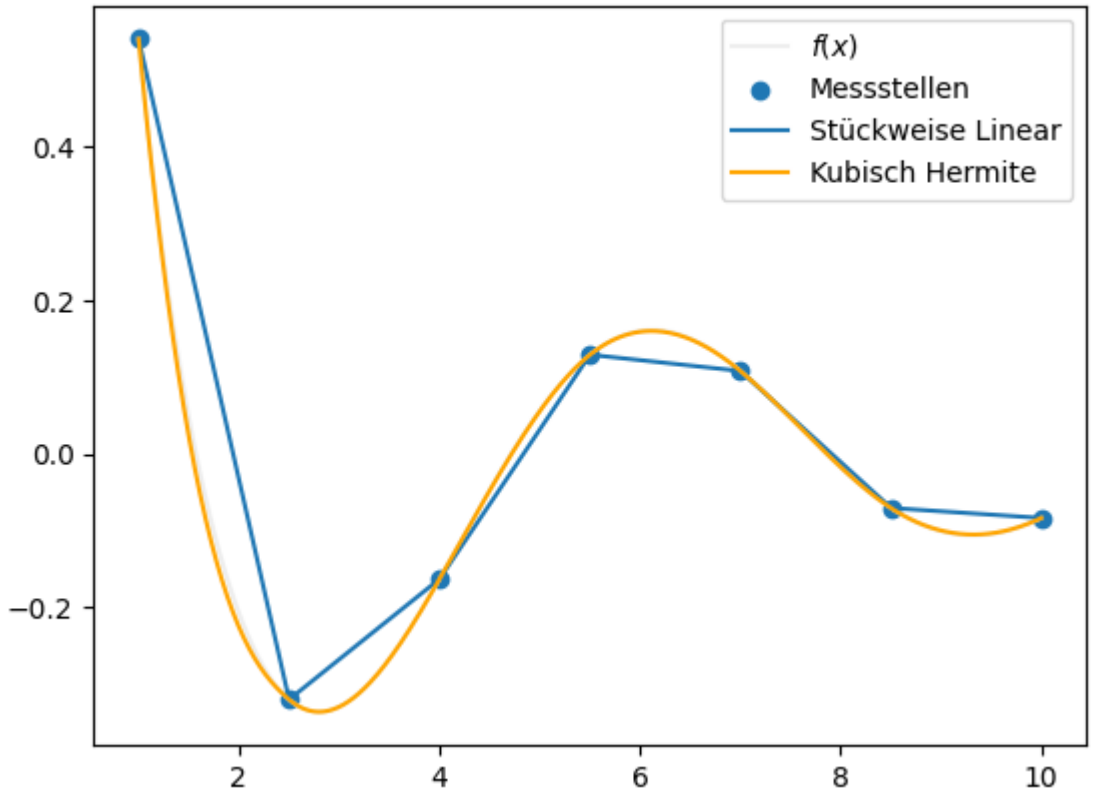
Es gilt:

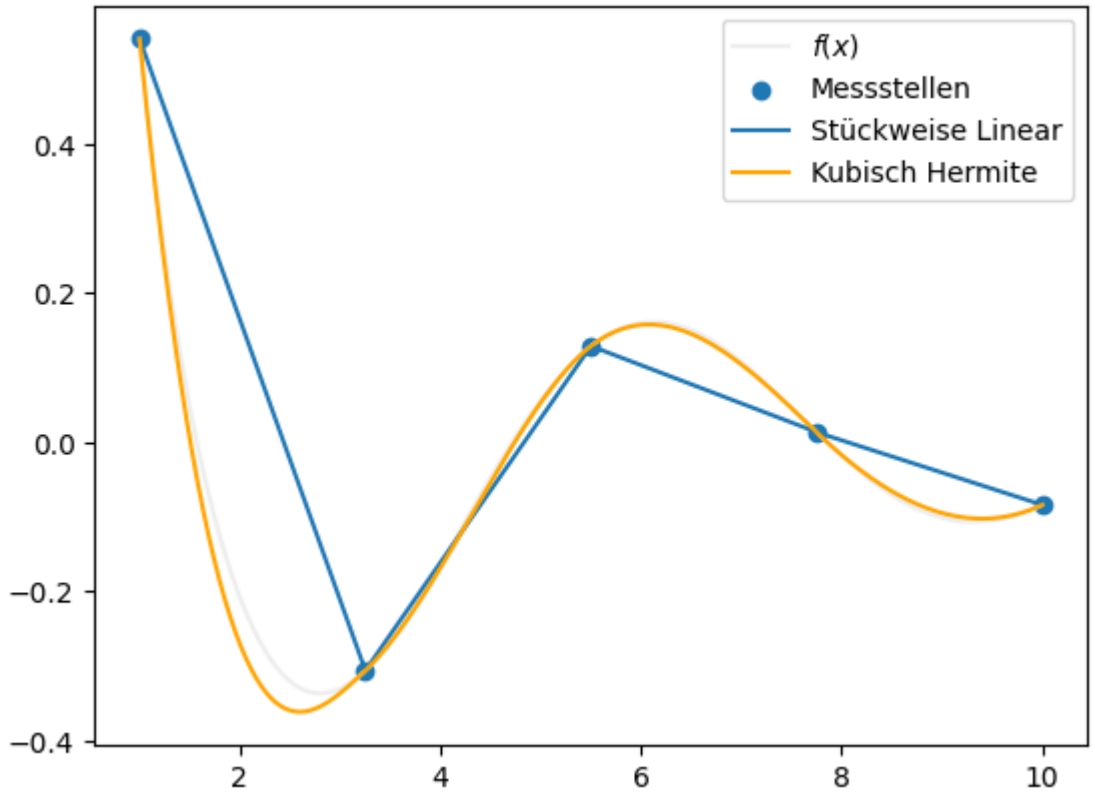
$$f_j(x_{j-1}) = y_{j-1} \quad f_j(x_j) = y_j$$

$$f'_j(x_{j-1}) = y'_{j-1} \quad f'_j(x_j) = y'_j$$

Also ist $\hat{f}(x) = f_j(x)$ in jedem $x_{j-1} \leq x < x_j$ eine stetige Funktion, die $f(x)$ annähert.

Kubisch Hermite: Plots





Splines

Splines: Definition

$$\mathcal{S}_{d,\mathcal{G}} = \left\{ s \in C^{d-1}[a,b], s_j := s_{|[x_{j-q}, x_j]} \text{ ist polynom von Grad} < d \right\}$$

Oder mit Worten:

*Stückweise definierte Aneinanderreihung von Polynomen, die an **allen** stellen stetig $(d-1)$-mal differenzierbar ist.*

Eine Spline der Ordnung 3 ist überall 2-mal differenzierbar!

Unterschied zu Hermite, die **nur einmal** differenzierbar sind!

Kubische splines: Grundlage

Denken wir zurück:

$$\phi(t) = 3t^2 - 2t^3 \qquad \psi(t) = t^3 - t^2$$

Hermite:

$$s_j(x) = s_{|[x_j-1,x_j]}(x) = y_{j-1}\phi(1-t) + y_j\phi(t) + dy_{j-1}h_j\psi(1-t) + dy_j h_j\psi(t)$$

Neue Knotenbedingungen:

$$s''_{|[x_{j-1},x_j]}(x_j) = s''_{|[x_j,x_{j+1}]}(x_j)$$

Anstatt dy_j als parameter zu nehmen, lösen wir ein Gleichungssystem nach $c_j = s'(x_j)$!

Kubische Splines: Gleichungssystem:

$$\frac{1}{h_j}c_{j-1} + \left(\frac{2}{h_j} + \frac{2}{h_{j+1}}\right)c_j + \frac{1}{h_{j+1}}c_{j+1} = 3\left(\frac{y_j - y_{j-1}}{h_j^2} + \frac{y_{j+1} - y_j}{h_{j+1}^2}\right)$$

Das entstandene lineare Gleichungssystem sieht so aus:

$$\begin{bmatrix} b_0 & a_1 & b_1 & 0 & \cdots & & \cdots & 0 \\ 0 & b_1 & a_2 & b_2 & & & & \\ & 0 & \ddots & \ddots & \ddots & & & \vdots \\ \vdots & & & \ddots & \ddots & \ddots & & \\ & & & & \ddots & a_{N-2} & b_{N-2} & 0 \\ 0 & \cdots & & \cdots & 0 & b_{N-2} & a_{N-1} & b_{N-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \\ \vdots \\ \\ c_{N-1} \\ c_N \end{bmatrix} = \begin{bmatrix} 3\left(\frac{y_1-y_0}{h_1^2} + \frac{y_2-y_1}{h_2^2}\right) \\ \\ \vdots \\ \\ 3\left(\frac{y_{N-1}-y_{N-2}}{h_{N-1}^2} + \frac{y_N-y_{N-1}}{h_N^2}\right) \end{bmatrix}$$

mit

$$b_j := \frac{1}{h_{j+1}}, \quad a_j := \frac{2}{h_j} + \frac{2}{h_{j+1}}$$

Splines: Freiheitsgrade

Wir haben $N + 1$ Knoten!

Anzahl **Stetigkeitsbedingungen**: $(N - 1)d$

(jeder Punkt ausser Endpunkte, für jede der $d - 1$ ableitungen plus die Funktion selber)

Anzahl **Polynomkoeffizienten**: $N(d + 1)$

(Jedes Liniensegment, polynome der Ordnung $d + 1$)

Anzahl **Freiheitsgrade**: $N(d + 1) - (N - 1)d = N + d$

(Freie Koeffizienten minus Bedingungen)

Kubische Spline: Freiheit

Für kobische Splines $(d = 3)$ gilt:

$$\dim \mathcal{S}_{d,\mathcal{G}} = N + 3$$

Gegebene Bedingungen:

$$N + 1 \text{ Punkte}$$

Verbleibende Freiheitsgrade:

$$2$$

Wir müssen je nach Anwendung **zwei weitere Bedingungen** festlegen!

Vollständige Kubische Spline-Interpolation

Um die zwei Freiheitsgrade zu füllen, wählen wir ein c_0 und c_N:

$$s'(x_0) = c_0, \qquad s'(x_N) = c_N$$

$$\begin{bmatrix} a_1 & b_1 & 0 & \cdots & \cdots & 0 \\ b_1 & a_2 & b_2 & & & \\ 0 & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ & & & \ddots & a_{N-2} & b_{N-2} \\ 0 & \cdots & \cdots & 0 & b_{N-2} & a_{N-1} \end{bmatrix} \begin{bmatrix} c_1 \\ \\ \vdots \\ \\ c_{N-1} \end{bmatrix} = \begin{bmatrix} 3\left(\frac{y_1-y_0}{h_1^2} + \frac{y_2-y_1}{h_2^2}\right) - c_0 b_0 \\ \\ \vdots \\ \\ 3\left(\frac{y_{N-1}-y_{N-2}}{h_{N-1}^2} + \frac{y_N-y_{N-1}}{h_N^2}\right) - c_N b_{N-1} \end{bmatrix}$$

Auch “Eingespannte” Kubische Spline

Natürliche Kubische Spline-Interpolation

Um die zwei Freiheitsgrade zu füllen, setzen wir $s''(x_0) = s''(x_N) = 0$:

$$\begin{bmatrix} 2 & 1 & 0 & \cdots & & & \cdots & 0 \\ b_0 & a_1 & b_1 & 0 & \cdots & & \cdots & 0 \\ 0 & b_1 & a_2 & b_2 & & & & \\ & 0 & \ddots & \ddots & \ddots & & & \vdots \\ \vdots & & & \ddots & \ddots & \ddots & & \\ & & & & \ddots & a_{N-2} & b_{N-2} & 0 \\ 0 & \cdots & & \cdots & 0 & b_{N-2} & a_{N-1} & b_{N-1} \\ 0 & \cdots & & & \cdots & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} c_0 \\ \vdots \\ c_N \end{bmatrix} = 3 \begin{bmatrix} \frac{y_1 - y_0}{h_1} \\ \frac{y_1 - y_0}{h_1^2} + \frac{y_2 - y_1}{h_2^2} \\ \vdots \\ \frac{y_{N-1} - y_{N-2}}{h_{N-1}^2} + \frac{y_N - y_{N-1}}{h_N^2} \\ \frac{y_N - y_{N-1}}{h_N} \end{bmatrix}$$

Auch “Eingespannte” Kubische Spline: Minimale Krümmung der gesamten Spline!

Periodische Kubische Spline-Interpolation

Wir wollen nun auch, dass $s'(x_0) = s'(x_N)$ und $s''(x_0) = s''(x_N)$:

$$\mathbf{A} := \begin{bmatrix} a_1 & b_1 & 0 & \cdots & 0 & b_0 \\ b_1 & a_2 & b_2 & & & 0 \\ 0 & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ 0 & & & \ddots & a_{N-1} & b_{N-1} \\ b_0 & 0 & \cdots & 0 & b_{N-1} & a_0 \end{bmatrix}$$

Wir lösen nur nach $c_0, \ldots, c_{N-1}$: Matrix ist quadratisch, rechte Seite des Systems bleibt Gleich!

Not-A-Knot Kubische Spline-Interpolation

s''' sei ebenfalls stetig in x_1 und x_{N-1}!

Bewirkt effektiv, dass $s_1 = s_2$ und $s_{N-1} = s_N$ - Deswegen “not-a-knot”

ETH zürich

Jonas Vonäsch
Teaching Assistant NumCS
jvonaesch@student.ethz.ch

jvonaesch.github.io